Case Study: How ShopKart Scaled Checkout

Engineering case study · ShopKart (fictional Indian e-commerce company) · Mega Monsoon Sale 2025

1. The problem

ShopKart is a mid-sized Indian e-commerce marketplace with about 9 million monthly active users. Every July it runs the **Mega Monsoon Sale**, a 72-hour event where traffic jumps to many times its normal level. During the 2024 sale, checkout collapsed twice. At the worst moment, 23 percent of checkout attempts failed, the payment page took up to 2.8 seconds at p99, and the company estimated it lost Rs. 4.1 crore in abandoned orders.

The post-mortem identified three root causes: the monolithic checkout service held database locks while waiting for the payment gateway; inventory was checked and decremented in the same transaction as order creation; and there was no rate limiting, so retry storms from mobile apps amplified every slowdown. Engineering leadership gave the newly formed checkout team, led by Meera Raghunathan, nine months to fix it before the 2025 sale.

2. Target architecture

The team broke checkout into four services connected by an internal message queue named **Saathi**, built on Apache Kafka. The key idea was to make the user-facing path as short as possible and push slow work to asynchronous consumers.

1. **Cart service** validates the cart and calculates the price.
2. **Inventory reservation service** places a temporary hold on stock.
3. **Order service** creates the order in a PENDING_PAYMENT state.
4. **Payment orchestrator** talks to payment gateways and publishes the result to Saathi; downstream consumers then confirm the order, release or commit inventory and send notifications.

3. Key design decisions

3.1 Inventory reservations instead of immediate decrement

Instead of decrementing stock during checkout, the inventory service creates a **reservation** that holds the item for **7 minutes**. If payment succeeds, the reservation is committed; if it fails or times out, a sweeper job releases it. The team chose 7 minutes after analysing 2024 data: 96 percent of successful UPI payments completed within 4 minutes, and the extra 3 minutes covered slower card OTP flows. Reservations are stored in Redis with a Lua script that checks and reserves stock atomically.

3.2 Idempotency everywhere

Mobile networks in India are unreliable, so clients retry. Without protection, a retry could create two orders or charge a customer twice. Every checkout request now carries an **idempotency key** generated by the app. The order service stores keys for **24 hours**; a repeated key returns the original response instead of creating a new order. Duplicate charges dropped from 312 cases in the 2024 sale to zero in 2025.

3.3 Rate limiting and load shedding

- A **token bucket** limiter at the API gateway allows each user **5 checkout requests per second** with a burst of 10.
- A global limiter caps total checkout traffic at **25,000 requests per second**, which load tests showed was 80 percent of safe capacity.
- When the global limit is hit, users are placed in a **virtual waiting room** with an estimated wait time instead of seeing an error.
- Mobile apps use exponential backoff with full jitter, starting at 200 ms and capped at 8 seconds.

3.4 Payment gateway resilience

ShopKart works with three payment gateways. Each gateway connection is wrapped in a **circuit breaker** that opens when the error rate exceeds 20 percent over a 30-second window, and traffic is routed to the next healthy gateway. During the 2025 sale the primary gateway's circuit opened four times, and customers did not notice any of them.

4. Load testing

Starting in April 2025 the team ran weekly load tests with a tool they called **Toofan**, which replayed anonymised 2024 sale traffic at up to three times its original volume. The tests found a connection-pool exhaustion bug in the order service in May, which would almost certainly have caused an outage. The final full-scale rehearsal was held on 6 July 2025, ten days before the sale.

5. Results

The 2025 Mega Monsoon Sale ran from 16 to 18 July. The table compares the most important metrics with the previous year.

Metric	2024 sale	2025 sale	Change
Peak orders per minute	6,200	18,400	~3x higher
Checkout p99 latency	2,800 ms	410 ms	85% lower
Checkout failure rate (worst hour)	23%	0.7%	-22.3 points
Duplicate charges	312	0	Eliminated
Inventory oversells	1,950 units	14 units	99% fewer
Peak infrastructure cost per hour	Rs. 3.6 lakh	Rs. 2.9 lakh	19% lower

Infrastructure cost fell despite triple the order volume, mainly because the asynchronous design let the team autoscale consumers independently and stop over-provisioning the monolith.

6. What did not go well

The remaining 14 oversold units came from a clock-skew bug: one sweeper node's clock was 9 seconds fast, so it released some reservations early. The fix was to compare reservation expiry against Redis server time instead of the local clock. The virtual waiting room also received complaints because its estimated wait times were too optimistic during the first hour of the sale.

7. Lessons learned

- Never hold a database lock while waiting on a third party.
- Separate reserving a resource from committing it.
- Make every write idempotent before you add retries.
- Shedding load politely is better than failing randomly.
- Load-test with real traffic shapes, not synthetic averages.